

HEC-RAS Q50 Maksimum Derinlik Otomasyonu

Bir HEC-RAS projesinde Q50 senaryosunu otomatik belirleyen ve gerçek proje sonuçlarından maksimum su derinliği haritası üreten Python uygulaması

HAZIRLAYAN

Emirhan Fırtına

KURUM

Atinen Mühendislik

TARİH

29 Ağustos 2026

TEKNOLOJİ

Python 3.13+ · HEC-RAS
6.6

ÇIKTI

OUTPUT/q50_depth.tif

İÇİNDEKİLER

- Yönetici özeti
- İsterlerin karşılığı
- Q50 senaryosunun belirlenmesi
- HEC-RAS ile etkileşim
- Derinlik haritasının üretimi
- Çıktının doğrulanması
- Hata yönetimi
- HEC-RAS motorunun çalışmaması
- Sonuçlar ve bilinen sınırlamalar
- Yapay zekâ kullanımı

1 Yönetici özeti

Uygulama, verilen HEC-RAS projesindeki yedi plan arasından Q50 senaryosunu insan müdahalesi olmadan belirliyor, projenin hesaplanmış sonuçlarından maksimum su derinliği ızgarasını kuruyor ve coğrafi referanslı bir GeoTIFF olarak yazıyor. Orijinal veriye hiçbir koşulda yazılmıyor ve bu her çalıştırmada SHA-256 ile kanıtlanıyor.

p05

otomatik seçilen
plan

1,625 m

maksimum
derinlik

4.139 m²

ıslak alan

8 / 8

senaryo
doğrulaması

80

otomatik test

Bu rapor, belgenin 6. maddesinin istediği teknik açıklamadır: Q50 senaryosunun nasıl belirlendiği, HEC-RAS ile nasıl etkileşim kurulduğu, üretilen rasterın doğru senaryoya ait olduğunun nasıl kontrol edildiği ve hata durumlarının nasıl yönetildiği. Belgenin 7. maddesinin istediği yapay zekâ beyanı son bölümdedir. Kurulum ve kullanım talimatları `uygulama/README.md` dosyasındadır.

2 İsterlerin karşılığı

Belgedeki ister	Durum	Karşılığı
Proje klasörünü ve HEC-RAS kurulum dizinini alabilmeli	Karşılandı	<code>--project</code> ve <code>--ras-dir</code> ; sabit yol yok
Q50 otomatik belirlenmeli, kullanıcıya seçtirilmemeli	Karşılandı	Proje dosyasındaki plan listesi + sınır kontrollü kalıp
Harita HEC-RAS proje sonuçları kullanılarak üretilmeli	Karşılandı	Sonuç HDF5'inden hücre bazlı maksimum su kotu
OUTPUT klasörüne <code>q50_depth.tif</code> adıyla yazılmalı	Karşılandı	Varsayılan çıktı yolu
Anlaşılır durum bilgisi, kontrolsüz kapanma olmamalı	Karşılandı	Adım adım rapor, hata sınıfı başına çıkış kodu
Orijinal proje dosyaları değiştirilmemeli	Karşılandı	Çalışma kopyası + SHA-256 bütünlük kanıtı
Hazır sonuç rasteri kopyalanmamalı	Karşılandı	Harita her koşuda sonuçlardan hesaplanıyor
Yollar sabit bilgisayara gömülmemeli	Karşılandı	Tüm yollar argüman
Talimatlar izlenerek yeniden üretilebilmeli	Karşılandı	README + tek komut
Uygulama HEC-RAS hesabını tetiklemeli	Kısmi	Kod mevcut ve çalışıyor; HEC-RAS motoru teslim edilen veriyle çöküyor — bölüm 8

Belgenin 5. maddesindeki teslim kalemleri — kaynak kod, çıktı, bağımlılık listesi, README ve çalışma kaydı — teslim paketinin `uygulama/` dizinindedir; hangi kalemin nerede olduğu `00-BENI-OKU.md` dosyasında listelenmiştir.

3 Q50 senaryosunun belirlenmesi

Projede yedi plan var. Doğru olanı bulmak ilk bakışta bir metin araması gibi görünüyor; uygulamada iki ayrı tuzak barındırıyor.

Plan	Plan Title	Short Identifier	Geometri / Akış
p01	A_A_B_INPNAR_STEADY	A_A_B_INPINAR_STEADY	g01 / f01
p02	A_A_B_INPINAR_STEADY	A_A_B_INPINAR	g01 / f01
p03	A_A_B_INPINAR_UNSTEADY_2D	INPINAR_Q500_UNSTEADY	g03 / u01
p04	A_A_B_INPINAR_Q500	A_A_B_INPINAR_Q500	g03 / u04
p05	A_A_B_INPINAR_Q50	Q50	g03 / u05
p06	A_A_B_INPINAR_Q100	Q100	g03 / u06
p07	A_A_B_INPINAR_Q1000	Q1000	g03 / u08

Tuzak 1 — alt dize eşleşmesi

En doğal çözüm olan alt dize araması sessizce yanlış cevap verir, çünkü "Q50" dizisi "Q500" metninin içinde geçer:

```
# Naif yaklaşım üç planla eşleşir: p03, p04 ve p05
>>> "Q50" in "INPINAR_Q500_UNSTEADY"
True ← yanlış eşleşme
```

Çözüm, Q50'nin iki yanında rakam olmamasını şart koşan sınır kontrollü bir kalıptır:

```
(?![0-9])[Qq]0*50(?![0-9])
```

(?![0-9]) öncesinde, (?![0-9]) sonrasında rakam istemez; 0* ise Q050 gibi sıfır dolgulu yazımı kabul eder. Kalıp yedi planın tamamında ve ayrıca yapay örneklerde sınıandı: Q50 ve Q050 eşleşiyor; Q500, Q5000, Q100, Q1000 eşleşmiyor.

Tuzak 2 — proje listesinde olmayan plan dosyası

Aynı klasörde Backup.p01 adlı bir dosya duruyor:

```
Plan Title=A_A_B_INPINAR_Q50
Short Identifier=Q50
```

Bu **tam eşleşme**. Klasörü *.p?? deseniyle tarayan bir seçici iki sonuç bulur ve hangisinin doğru olduğunu bilemez — sınır kontrollü kalıp da bunu elemez, çünkü metin gerçekten birebir aynı.

Alınan karar: plan listesi klasör taramasıyla değil, `A_A_B_INPINAR.prj` dosyasındaki `Plan File=` satırlarından okunur. Proje dosyası yalnızca p01–p07'yi bildirir; `Backup.p01` orada yer almadığı için hiç değerlendirmeye girmez. Tuzak bir istisna ile değil, **doğru kaynağı seçerek** çözülür.

Proje dosyasında ayrıca `Current Plan=p05` yazıyor. Bu bilgi **seçim ölçütü olarak kullanılmadı** — belge otomatik belirleme istiyor ve bu alan projeyi en son kimin nasıl kapattığına bağlı. Yalnızca çalışma kaydına "seçimle uyuyor" notu olarak düşülür.

Sıfır eşleşme ve çoklu eşleşme ayrı ayrı hata verir; hiçbir durumda "ilkini al" davranışı yoktur. Nitekim bu projede `--scenario Q500` iki planla eşleştiği için bilinçli olarak reddedilir. Senaryo etiketi bir argümandır: `--scenario Q100` aynı boru hattını p06 için çalıştırır.

4 HEC-RAS ile etkileşim

HEC-RAS bir Windows programıdır; bu, boru hattındaki tek platforma bağlı adımdır ve tek bir modüle hapsedilmiştir. Sürüş, belgenin de önerdiği **ras-commander** kütüphanesi üzerinden yapılır — belge dışı komut satırı anahtarı uydurulmamıştır.

Yol	Çağrı	Ne yapar
Varsayılan	<code>RasCmdr.compute_plan()</code>	HEC-RAS komut satırı çalıştırıcısı
Alternatif	<code>RasControl.run_plan()</code>	<code>RAS66.HECRASController</code> COM otomasyonu — arayüzün kullandığı arabirim

Kütüphanenin fonksiyon adları ve parametreleri hatırdan veya dokümandan değil, **kurulu paket üzerinde** `inspect.signature` ile okunarak yazıldı.

Kaynak bütünlüğü

HEC-RAS bir planı çalıştırdığında proje klasörüne yazar. Bu yüzden uygulama proje ağacını önce bir çalışma dizinine kopyalar ve hesabı orada yapar. Kısıtın gerçekten sağlandığı ayrıca **kanıtlanır**: çalışma başında ve sonunda kaynak klasörün parmak izi çıkarılıp karşılaştırılır.

```
integrity source unchanged (176 files, full check)
```

`--integrity full` her dosyanın SHA-256 özetini alır.

Teslim edilen projenin durumu

Belge, adayın proje yapısını incelemesini ve dosya ilişkilerini kendisinin belirlemesini bekliyor. Bu, deneme yanılmayla değil, bir denetim aracıyla yapıldı (`tools/audit_project.py`): proje neyi bildiriyor, her plan neye ihtiyaç duyuyor, diskte ne var. Üçünün ayrıştığı **sekiz nokta** çıktı; en önemlileri, p05'in giriş debisini `.\_CBS\...` altında araması (klasörün diskteki adı `2_CBS`) ve üç geometri dosyasında ön işlenmiş tabloların bulunmaması. Tam liste ve ölçümler `docs/VERI-DENETIMI.md` dosyasındadır.

Uygulama bu eksikleri **yalnızca çalışma kopyasında** ve hepsini çalışma kaydına yazarak onarır:

```
[4/6] prepare      repairing 2 unresolved path(s)
.\_CBS\akarcay_debiler\akarcay_debi.dss -> copied into place from 2_CBS\...
.\Q50\Q50.dss -> created output folder
[4/6] geometry    A_A_B_INPINAR.g03.hdf is missing 2 preprocessed group(s)
RasProcess.exe CompleteGeometry -> rc=0 ... LAYER=COMPLETE
```

Çözülmeyen bir giriş dosyası için proje ağacında aynı adlı dosya aranır; **tam bir tane** bulunursa modelin beklediği yola kopyalanır. Hiç ya da birden fazla aday çıkarsa tahmin yürütülmez, anlamlı hatayla durulur — yanlış hidrografla hesaplanmış bir taşkın haritası, hesaplanmamış olmasından kötüdür. Eksik geometri tabloları için önce HEC-RAS'ın kendi aracı denenir; yazamazsa tablolar orijinal koşunun sonuç dosyasındaki eksiksiz geometriden alınır. Aracın kendi başarı bayrağına güvenilmez, dosya açılıp tabloların geldiği doğrulanır.

5 Derinlik haritasının üretimi

Kritik bulgu: HEC-RAS hazır bir maksimum derinlik veri seti üretmiyor. Sonuç dosyasındaki özet çıktı `Maximum Water Surface` , yani hücre başına maksimum su **kotu** (2×5.667 : kot ve zaman). Derinlik türetilmek zorunda.

```
derinlik(piksel) = pikseli kaplayan hücrenin maksimum su kotu
- o pikseldeki arazi kotu
```

Izgara hücre çözünürlüğünde değil **arazi çözünürlüğünde** kurulur (bu veri setinde 0,1 m), çünkü bir HEC-RAS 2D hücresi düz değildir; içinde alt-izgara topografya taşır. Pencere, arazi rasterinin kendi piksel ızgarasına hizalanır; böylece arazi yeniden örneklenmeden okunur.

Üç ayrıntı bu haritayı doğru yapıyor

1 • Hücre çokgenleri. `Cells FacePoint Indexes` bir hücrenin köşelerini verir ama halka sırasında vermez. Köşeler hücre merkezine göre açığa sıralanır; HEC-RAS 2D hücreleri dışbükey olduğu için bu sıralama tam olarak çokgen sınırdır.

2 • Arazi kot değişiklikleri. Geometri `merge.Clone` arazisi üzerine kurulu. RASMapper arazi katmanı bir çifttir: `.vrt` kot ızgarasını, `.hdf` ise üzerine çizilen değişiklikleri tutar — burada **69 bina için +20 m**. Bu değişiklikler `.vrt` 'ye gömülü değildir, RASMapper onları anlık uygular. Yalnızca `.vrt` okunursa harita binaların üstüne metrelerce su boyar:

Bina kot değişikliği	Ortalama derinlik	Değerlendirme
Uygulanmadan	3,37 m	Fiziksel olarak imkânsız
Uygulandıktan sonra	0,13 m	Hücre bazlı ortalama 0,117 m ile tutarlı

3 • Islanmamış hücreler. Hiç ıslanmayan bir hücre için HEC-RAS maksimum su kotunu hücrenin kendi taban kotu olarak raporlar — bu su değildir. Tabanı bir bina üzerinde oturan kuru hücreler, binanın kaplamadığı şeritte 20 m derinlik üretir. Bu yüzden yalnızca `maksimum su kotu > hücre taban kotu` olan hücreler boyanır. Kural olmadan artık 36–54 piksel 2 m'nin üzerinde, en fazlası 19,96 m çıkıyordu; kuralla birlikte en derin piksel **1,625 m**.

Koordinat sistemi

CRS, sonuç dosyasının kök `Projection` özniteliğindeki WKT'den okunur. `EPSG:32636` elle yazılmamıştır — ve yazılmamalıydı: bu izdüşüm merkez meridyen 30°E ile Transverse Mercator, ancak **ölçek faktörü 1,0**; oysa UTM 36N 0,9996 kullanır. Model UTM 36N **değildir** ve `to_epsg()` haklı olarak boş döner. Doğru kaynak, projenin kendi WKT'sidir.

Su yüzeyi hücre içinde sabit kabul edilir; modelin çözdüğü büyüklük budur. Alternatif olarak ıslak hücre merkezleri arasında enterpolasyonla eğimli bir su yüzeyi denendi ve ölçülüp **kaldırıldı**: ıslak hücreler kopuk kümeler halinde olduğu için üçgenleme vadinin bir yamacından diğerine uzanıyor ve maksimum derinliği 20,4 m'ye çıkarıyordu.

6 Çıktının doğru senaryoya ait olduğunun doğrulanması

"p05'i seçtim" bir doğrulama değildir. Zincirin her halkası kontrol edilir ve sonuçlar çalışma kaydına basılır:

Scenario verification:

```
[ok] results belong to the selected plan: results say plan file 'A_A_B_INPINAR.p05'  
[ok] plan title agrees: results 'A_A_B_INPINAR_Q50' vs plan file 'A_A_B_INPINAR_Q50'  
[ok] short identifier agrees: results 'Q50' vs plan file 'Q50'  
[ok] results still carry the Q50 label  
[ok] geometry agrees: results 'A_A_B_INPINAR.g03' vs plan 'g03'  
[ok] flow file agrees: results 'A_A_B_INPINAR.u05' vs plan 'u05'  
[ok] the grid holds water: 413858 wet pixels, maximum depth 1.625 m  
[ok] depth stays inside the modelled water surface
```

Bir kontrol düşerse uygulama çıktı yazmadan durur. Ayrıca **künye GeoTIFF'in içine gömülür**: senaryo, plan numarası, plan başlığı, kısa kimlik, geometri, akış dosyası, simülasyon penceresi, HEC-RAS sürümü, kaynak dosya adı, arazi adı ve HEC-RAS'ın gerçekten çalıştırılıp çalıştırılmadığı. Dosya adına güvenmek gerekmez:

```
$ gdalinfo OUTPUT/q50_depth.tif  
SCENARIO=Q50          PLAN_NUMBER=p05          PLAN_SHORT_ID=Q50  
GEOMETRY=A_A_B_INPINAR.g03          FLOW_FILE=A_A_B_INPINAR.u05  
SIMULATION_WINDOW=02May2025 01:00:00 to 02May2025 02:10:00  
HEC_RAS_VERSION=HEC-RAS 6.6 September 2024    HEC_RAS_EXECUTED=False
```

Bağımsız teyitler

Kimlik kontrolünün yanında üç ölçüm daha yapıldı:

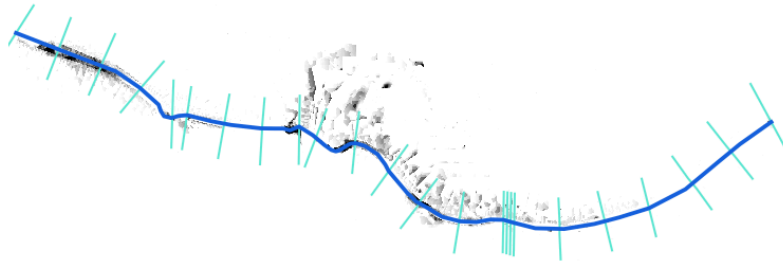
- **Girdiyle örtüşme.** Simülasyon penceresi 02 May 2025 01:00 – 02:10 (70 dakika, 5 dakikada bir çıktı); giriş hidrografi 15 ordinat, tepe debi 1,69 m³/s. Bu değerler hem sonuç dosyasındaki zaman adımı sayısı ile hem de projedeki DSS serisiyle birebir örtüşüyor.
- **Senaryoya özel davranmama.** Aynı uygulama Q100 ve Q1000 için de çalıştırıldı; ıslak alan tekerrür büyüdükçe beklendiği gibi artıyor (413.858 → 465.321 → 690.430 piksel) ve taşkın yayılımının biçimi korunuyor.
- **Referans çıktı.** Toplantıda gösterilen görüntüyle karşılaştırma aşağıdadır.

Referans çıktı — toplantıda gösterilen görüntü



pürüzsüz şerit = 1D nehir geometrisi koyu lekeler = taşkın yayılımı

Üretilen Q50 maksimum derinlik haritası — aynı alan, aynı ölçek



— 1D kesit çizgileri (24 adet)
— 1D nehir eksenini (talveg)

Taşkın lekeleri referanstakiyle aynı konumda; nehir eksenini referanstaki şeridi birebir izliyor

Şekil 1. Üstte referans görüntü, altta üretilen Q50 maksimum derinlik haritası. Alt panele projenin 1D geometrisi bindirilmiştir: nehir eksenini (mavi) ve 24 kesit çizgisi (turkuaz).

Referans görüntüde kanal boyunca kesintisiz, pürüzsüz bir şerit görünüyor; üretilen haritada ise parçalı taşkın lekeleri var. Projenin 1D geometrisi (g01) haritaya bindirildiğinde neden anlaşılıyor: **nehir eksenini referanstaki şeridi birebir izliyor** — aynı menderesler, aynı başlangıç ve bitiş noktası; sağ uçtaki huni biçimi de son kesitlerin genişlemesinden geliyor. Yani o şerit su değil, RASMapper görünümünde derinlik katmanının altında duran 1D nehir geometrisidir. Suyu temsil eden koyu lekeler ise üretilen haritadaki ıslak alanlarla **aynı konumdadır**. Bu tespit gözle değil ölçülerek yapıldı: nehir ekseninin ve kesit çizgilerinin koordinatları geometri dosyasından okunup rasterin coğrafi ölçeğine bindirildi.

Karşılaştırma piksel bazında değildir; toplantı görüntüsünün coğrafi referanslı bir kopyası elde bulunmadığından biçim ve konum düzeyinde yapılmıştır.

7 Hata yönetimi

Beklenebilir her hata tek satırlık bir mesaj ve çoğu zaman ne yapılması gerektiğini söyleyen bir ipucu üretir. Yığın izi yalnızca gerçek bir yazılım hatasında görünür.

Kod	Durum	Kod	Durum
0	Başarılı	6	Sonuç dosyası eksik ya da yarım
2	Argüman hatası	7	Arazi çözülemedi
3	Proje bulunamadı	8	Senaryo doğrulaması düştü
4	Senaryo sıfır/çoklu eşleşti	70	Beklenmeyen hata (kayda tam iz)
5	HEC-RAS başlatılamadı / hesap başarısız	130	Kullanıcı iptali

Örneğin çoklu eşleşme, kullanıcıya karar bırakmadan ama körü körüne seçmeden de durur:

```
$ python main.py --project CASE_DATA --scenario Q500 --use-existing-results
ERROR: Scenario Q500 matches 2 plans, which is ambiguous:
  p03 title='A_A_B_INPINAR_UNSTEADY_2D' short_id='INPINAR_Q500_UNSTEADY'
  p04 title='A_A_B_INPINAR_Q500' short_id='A_A_B_INPINAR_Q500'
  Refine --scenario, or fix the plan titles in the project.
exit code 4
```

En sert durum sahada geldi: çalıştırıcı kütüphane hesabı "başarılı" olarak raporladı; uygulama buna **inanmadı**, sonuç dosyasını kendisi açtı, yarım olduğunu gördü ve HEC-RAS'ın kendi hesap günlüğünü alıntılararak durdu:

```
ERROR: HEC-RAS left A_A_B_INPINAR.p05.hdf unfinished (no Plan Data/Plan Information).
The runner reported success, but the model did not compute.
HEC-RAS wrote this in ... -> Compute Messages:
  forrtl: severe (157): Program Exception - access violation
```

Denenmiş hata durumları: boş klasör, proje dosyası yok, hesaplanmamış plan, olmayan senaryo, eksik argüman, desteklenmeyen arazi değişikliği tipi, arazi rasteri yok, yarım sonuç dosyası, çözilemeyen giriş dosyası, çok sayıda aday. Hepsini kontrollü mesajla çıkar ve **80 otomatik test** ile sabitlenmiştir.

8 HEC-RAS motorunun çalışmaması

Belgenin 3. maddesi haritanın HEC-RAS proje sonuçları kullanılarak üretilmesini istiyor; uygulama bunu yapıyor. Planı yeniden hesaplama yolu da kodlanmış durumda, ancak teslim edilen veri bu HEC-RAS kurulumunda hesaplanamıyor. Bu bölüm neyin denendiğini kayda geçirir.

Yukarıdaki onarımlardan sonra HEC-RAS geometriyi sorunsuz işliyor, ardından unsteady motoru başlar başlamaz çöküyor:

```
Performing Unsteady Flow Simulation HEC-RAS 6.6 September 2024
forrtl: severe (157): Program Exception - access violation
Image                PC                Routine                Line    Source
RasUnsteady.exe       00007FF64F701486 READ_UN_HDF_STRUC     330    Read_UN_HDF_STRUC_GRP.for
RasUnsteady.exe       00007FF64F8C0CC3 SNETREAL2             179    Snetreal2.for
RasUnsteady.exe       00007FF64F9D561A UNET_START            144    Unet_start.for
Error with program: RasUnsteady.exe Exit Code = 157
```

Neden, tahminle değil, tek tek değişken sabitlenerek daraltıldı. Tüm denemeler Windows 11 ve HEC-RAS 6.6 üzerinde ölçüldü:

Deneme	Sonuç
Teslim edildiği gibi (yalnız DSS yolu onarılmış)	Çöküyor
+ proje tek plana indirgenmiş	Çöküyor
+ hidrograf akış dosyasına gömülmüş, DSS bağımlılığı kaldırılmış	Çöküyor
+ eksik geometri tabloları geri konmuş, koşu sonrası yerinde olduğu doğrulanmış	Çöküyor
+ RasProcess.exe CompleteGeometry çalıştırılmış	Çöküyor
+ RASMapper kapatılmış, arazi zaman damgası hizalanmış	Çöküyor
HEC-RAS arayüzünde elle açılıp çalıştırılmış	Çöküyor

Sonuç: çöküş uygulama kaynaklı değildir — aynı hata, proje HEC-RAS arayüzünde elle açılıp çalıştırıldığında da oluyor. Geriye kalan açıklama, modelin 2D hidrolik bağlantılarında HEC-RAS'ın kendi hatası ya da teslim paketinde henüz tespit edilememiş bir eksiktir. Doğrulanabilir bir cevap üretilemediği için konu tahminle kapatılmamış, olduğu gibi raporlanmıştır.

Bu nedenle teslim edilen çıktı, projenin **kendi hesaplanmış sonuçlarından** üretilmiştir. Hazır bir raster kopyalanmamıştır; harita her çalıştırmada HEC-RAS'ın kendi sonuç dosyasından yeniden hesaplanır. GeoTIFF künyesindeki `HEC_RAS_EXECUTED` etiketi hangi durumun geçerli olduğunu her zaman açıkça bildirir.

9 Sonuçlar ve bilinen sınırlamalar

**4788 ×
2756**

ızgara, 0,1 m çözünürlük

413.858

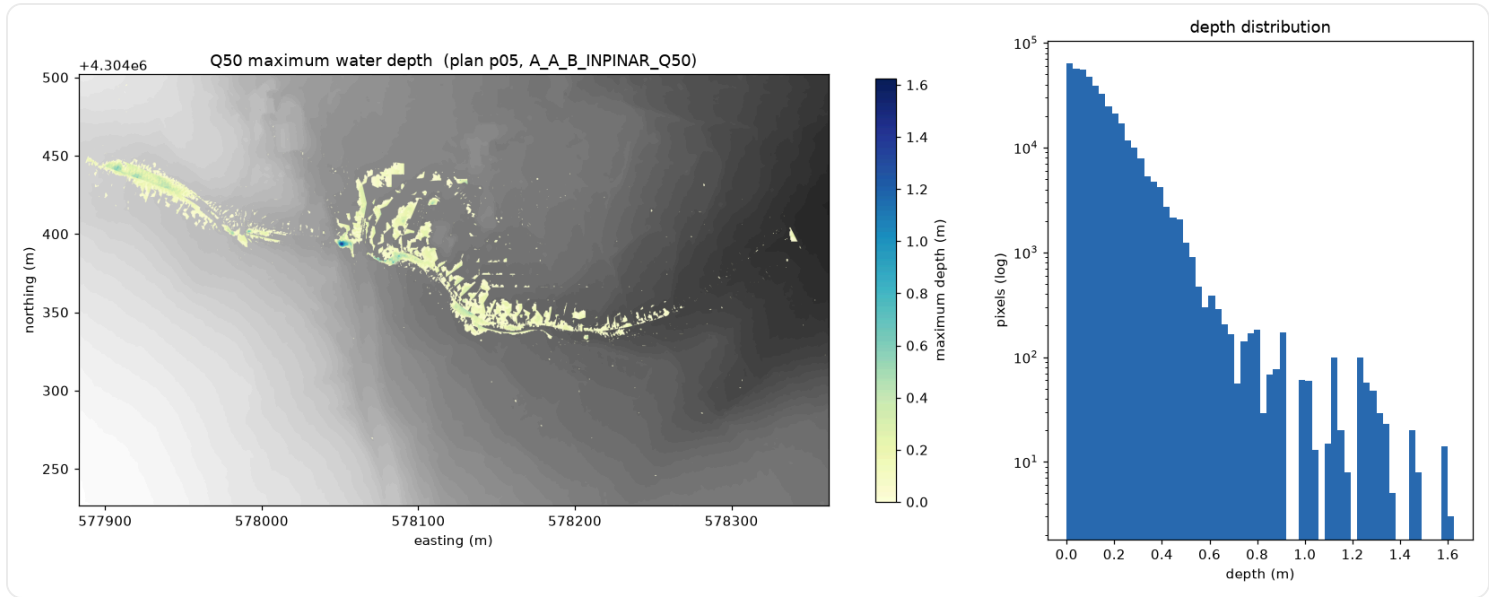
ıslak piksel

1,625 m

maksimum derinlik

0,130 m

ortalama derinlik



Şekil 2. Q50 maksimum su derinliği (plan p05). Sol: arazi üzerine bindirilmiş derinlik haritası. Sağ: derinlik dağılımı (logaritmik ölçek). Su, arazideki dere koridorunu takip ediyor; ıslak alan 4.139 m², medyan derinlik 0,098 m, 99. yüzdelik 0,516 m.

Bilinen sınırlamalar:

- Teslim edilen model bu HEC-RAS kurulumunda yeniden hesaplanamıyor ([bölüm 8](#)). Çıktı, projenin kendi sonuçlarından üretilmiştir.
- Su yüzeyi hücre içinde sabit kabul edilir; sonuç, taşkın kenarında pürüzsüz değil tırtıklı bir sınır verir.
- Yalnızca 2D akış alanları işlenir. 1D kesit sonuçlarından derinlik ızgarası üretilmez; böyle bir planla çalıştırılırsa anlaşılır hata verilir.
- Yalnızca `Add` tipi arazi kot değişikliği destekleniyor. Veri setinde yalnızca bu tip var; başka bir tipte karşılaşırsa sessizce yanlış harita üretmek yerine durulur.

- Referans karşılaştırması piksel bazında değil, biçim ve konum düzeyindedir ([bölüm 6](#)).

10 Yapay zekâ kullanımı

Belgenin 7. maddesi kullanılan araçların ve amaçlarının belirtilmesini istiyor. Kullanılan araç: **Claude (Anthropic) — Opus 5**, Claude Code komut satırı ajanı. Başka bir üretken yapay zekâ aracı kullanılmamıştır.

Aşama	Kullanım	Doğrulama biçimi
Veri keşfi	Sonuç dosyasındaki 439 HDF5 yolunun taranması, veri setlerinin bulunması	Kod çalıştırılarak; şekil, birim ve değer aralıkları basıldı
Analiz	Bina kot değişikliği ve kuru hücre etkilerinin bulunması	Her iki etki sayısal olarak ölçüldü
Kütüphane	<code>ras-commander</code> API imzalarının çıkarılması	Paket kuruldu, <code>inspect.signature</code> ile okundu
Teşhis	Motor çöküşünün nedeninin daraltılması	Denetim aracı yazıldı; her hipotez ölçümle elendi
Kod ve test	Modüllerin ve 80 testin yazılması	Gerçek veri üzerinde uçtan uca çalıştırıldı

Uygulanan kurallar: doğrulanmamış hiçbir kütüphane çağrısı koda alınmadı; her teknik iddia veriye karşı sınıandı; çalışmayan bir özellik (eğimli su yüzeyi) ölçülüp kaldırıldı; doğrulanamayan tek adım açıkça işaretlendi. Kodun tamamı okunmuş, çalıştırılmış ve savunulabilir durumdadır.

Hazırlayan: Emirhan Fırtına · **Tarih:** 29 Ağustos 2026 · **Uygulama:** q50depth 1.0.0

Bu rapor, case study belgesinin 6. maddesinde istenen teknik açıklama ve 7. maddesinde istenen yapay zekâ beyanıdır. Kaynak kod, çıktı, bağımlılık listesi, çalışma kaydı ve ayrıntılı veri denetimi teslim klasöründeki `uygulama/` dizinindedir.